

BMAD em Times Enterprise Pequenos

Guia Estratégico e Operacional para Uso de IA no Desenvolvimento de Software

Sumário

1. Introdução
 2. O que é BMAD
 3. Objetivos do BMAD
 4. Quando BMAD faz sentido
 5. O papel da IA dentro do BMAD
 6. Modelo operacional recomendado
 7. Fluxo ideal entre os agentes BMAD
 8. PRD — Product Requirements Document
 9. ADR — Architecture Decision Record
 10. OpenAPI como contrato central
 11. Context Engineering
 12. Organização de repositórios
 13. Estratégia Monorepo
 14. Estratégia Multirepo
 15. Context Repository
 16. Estrutura ideal dos projetos
 17. Estrutura de contexto para IA
 18. Fluxo operacional do time
 19. Integração com pipelines corporativas
 20. Melhores práticas
 21. O que evitar
 22. Recomendações finais
-

1. Introdução

Este documento consolida as melhores práticas para adoção do BMAD (BMad Method) em ambientes corporativos, especialmente em tiny teams compostos por:

- PM
- Backend
- Frontend
- QA

O foco principal é:

- uso estratégico de IA

- compartilhamento de contexto
 - integração entre frontend e backend
 - arquitetura sustentável
 - rastreabilidade
 - produtividade com qualidade
-

2. O que é BMAD

BMAD é uma metodologia de desenvolvimento orientada por IA baseada em:

- agentes especializados
- contexto estruturado
- documentação versionada
- contratos compartilhados
- fluxo colaborativo

O objetivo NÃO é substituir pessoas, mas:

- ampliar a capacidade do time
 - reduzir perda de contexto
 - aumentar consistência
 - acelerar execução
-

3. Objetivos do BMAD

O BMAD busca resolver problemas comuns em times pequenos:

- contexto espalhado
 - documentação inexistente
 - dependência de pessoas-chave
 - desalinhamento entre frontend e backend
 - perda de decisões arquiteturais
 - retrabalho
 - inconsistência técnica
-

4. Quando BMAD faz sentido

BMAD funciona muito bem para:

- tiny teams
- produtos complexos
- APIs enterprise
- times altamente técnicos
- sistemas com muitas regras de negócio

- integração forte entre frontend e backend
-

5. O papel da IA dentro do BMAD

Melhor modelo atual

Humanos:

- estratégia
- domínio
- arquitetura
- validação

IA:

- aceleração
 - boilerplate
 - documentação
 - análise
 - auxílio operacional
-

6. Modelo operacional recomendado

Human-led AI Development

Humano define direção → IA acelera execução → Humano valida

7. Fluxo ideal entre os agentes BMAD

Discovery → PRD → ADR → OpenAPI → Backend + Frontend → QA → Deploy

Analyst Agent

Responsável por:

- entendimento do domínio
 - escopo
 - regras iniciais
 - fluxos
-

PM Agent

Responsável por:

- PRD
 - histórias
 - critérios de aceite
-

Architect Agent

Responsável por:

- arquitetura
 - contratos
 - ADRs
 - decisões técnicas
-

Backend Agent

Responsável por:

- APIs
 - services
 - integração
 - persistência
-

Frontend Agent

Responsável por:

- telas
 - UX
 - integração frontend/backend
-

QA Agent

Responsável por:

- cenários
 - validações
 - automações
-

8. PRD — Product Requirements Document

Documento funcional que descreve:

- problema
- objetivos
- regras

- requisitos
- comportamento esperado

Estrutura recomendada

1. Objetivo
2. Problema
3. Escopo
4. Regras de negócio
5. Fluxos
6. Critérios de aceite
7. Requisitos funcionais
8. Requisitos não funcionais

9. ADR — Architecture Decision Record

Documento responsável por registrar:

- decisões arquiteturais
- contexto
- tradeoffs
- impactos

Estrutura recomendada

1. Título
2. Status
3. Contexto
4. Decisão
5. Consequências
6. Alternativas consideradas
7. Referências

10. OpenAPI como contrato central

Melhor prática

OpenAPI deve ser:

- centralizado
- versionado
- compartilhado
- definido antes da implementação

Fluxo ideal:

Regras → Contratos → Backend → Frontend → QA

11. Context Engineering

Context Engineering é o processo de:

- estruturar conhecimento
 - versionar contexto
 - compartilhar decisões
 - alimentar IA corretamente
-

12. Organização de repositórios

Existem dois modelos principais:

- Monorepo
 - Multirepo
-

13. Estratégia Monorepo

Estrutura recomendada

```
produto/
├── apps/
│   ├── backend/
│   └── frontend/
└── docs/
    ├── contracts/
    ├── adr/
    └── .ai/
```

Melhor prática

- um único Git repository
- pipelines separadas por path

Evitar:

- múltiplos .git internos
 - nested repositories
-

14. Estratégia Multirepo

Estrutura recomendada

```
produto-context/  
produto-backend/  
produto-frontend/
```

Melhor prática

Utilizar:

- context repo centralizado
- workspace compartilhado

15. Context Repository

Objetivo

Ser:

- cérebro do produto
- fonte da verdade
- camada compartilhada de contexto

Estrutura recomendada

```
docs/  
contracts/  
adr/  
.ai/  
flows/  
stories/
```

16. Estrutura ideal dos projetos

```
workspace/  
├─ trusthub-context/  
├─ trusthub-backend/  
└─ trusthub-frontend/
```

17. Estrutura de contexto para IA

```
.ai/  
├── shared/  
├── backend/  
└── frontend/
```

Exemplos

```
.ai/shared/company-standards.md  
  
.ai/backend/backend-agent.md  
  
.ai/frontend/angular-guidelines.md
```

18. Fluxo operacional do time

Melhor prática

Macro:

- fluxo sequencial

Micro:

- trabalho paralelo e colaborativo

19. Integração com pipelines corporativas

Melhor prática enterprise

Manter:

- pipelines independentes
- deploy separado
- repositórios separados

Adicionar:

- context repo compartilhado
- OpenAPI centralizado

- workspace compartilhado
-

20. Melhores práticas

✓ OpenAPI First ✓ Context-first development ✓ ADRs curtos e objetivos ✓ PRD versionado ✓ IA contextualizada ✓ Contratos centralizados ✓ Padrões corporativos compartilhados

21. O que evitar

✗ IA trabalhando sem contexto ✗ OpenAPI apenas no backend ✗ Regras fora do Git ✗ Documentação desconectada ✗ Duplicação manual de contexto ✗ Nested repositories complexos

22. Recomendações finais

Melhor modelo para o cenário analisado

Multi repo + Context repo compartilhado + Workspace compartilhado + OpenAPI centralizado

Conclusão

BMAD entrega mais valor quando:

- humanos lideram
- IA acelera
- contexto é estruturado
- contratos são compartilhados
- arquitetura é rastreável
- padrões corporativos são respeitados